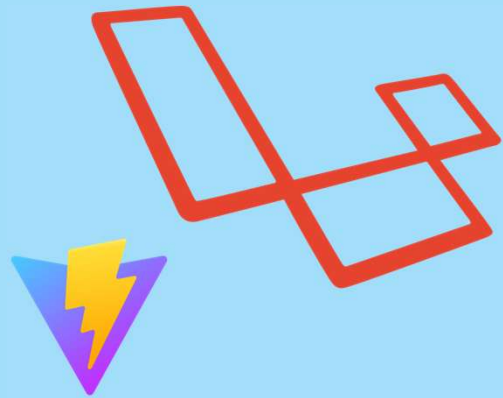




UNIVERSIDAD
**PABLO
OLAVIDE**
SEVILLA



Laravel



Tecnologías Avanzadas de Desarrollo

UD3: Frameworks – Laravel Básicos

Olga M. Moreno Martín

Licencia



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Índice



Laravel

1. Instalación en entorno local
2. Migraciones
3. Carpetas principales
4. Rutas
5. Estructuras de control y plantillas
6. Modelos y relaciones con Eloquent

Laravel

Para instalar Laravel, necesitamos su gestor de paquetes, Composer.

<https://getcomposer.org/download/>

 [Home](#) | [Getting Started](#) | [Download](#) | [Documentation](#) | [Browse Packages](#)

Download Composer Latest: v2.9.5

 **PRIVATE PACKAGIST**
Secure & reliable package repository

Windows Installer

The installer - which requires that you have PHP already installed - will download Composer for you and set up your PATH environment variable so you can simply call `composer` from any directory.

Download and run [Composer-Setup.exe](#) - it will install the latest composer version whenever it is executed.

Command-line installation

To quickly install Composer in the current directory, run the following script in your terminal. To automate the installation, use [the guide on installing Composer programmatically](#).

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php -r "if (hash_file('sha384', 'composer-setup.php') === 'c8b085408188070d5f52bcfe4ecfbee5f727afa458b2573b8eae
php composer-setup.php
php -r "unlink('composer-setup.php');"
```

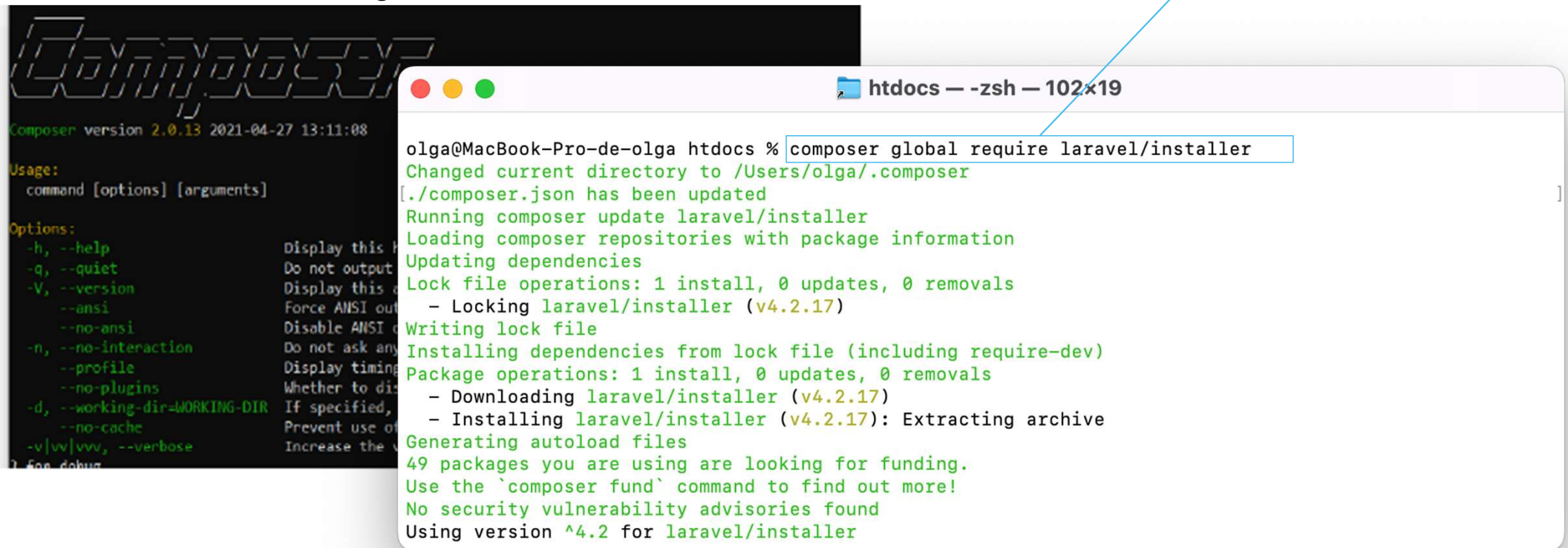
This installer script will simply check some `php.ini` settings, warn you if they are set incorrectly, and then download the latest `composer.phar` in the current directory. The 4 lines above will, in order:

- Download the installer to the current directory
- Verify the installer SHA-384, which you can also [cross-check here](#)
- Run the installer
- Remove the installer

Most likely, you want to put the `composer.phar` into a directory on your PATH, so you can simply call `composer` from any directory (*Global install*), using for example:  Olga M. Moreno Martín

Instalación en local

Una vez instalado composer abrimos la terminal, verificamos la instalación introduciendo `composer` como instrucción y en la carpeta que queramos vamos a instalar Laravel en global.



```
olga@MacBook-Pro-de-olga htdocs % composer global require laravel/installer
Changed current directory to /Users/olga/.composer
[./composer.json has been updated]
Running composer update laravel/installer
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking laravel/installer (v4.2.17)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Downloading laravel/installer (v4.2.17)
  - Installing laravel/installer (v4.2.17): Extracting archive
Generating autoload files
49 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found
Using version ^4.2 for laravel/installer
```

Instalación en local

Nombre de nuestro proyecto

Versión

`composer create-project laravel/laravel nuevo "11.*"`

```
htdocs — php — 102x28
sh-3.2# composer create-project laravel/laravel example-app
Do not run Composer as root/super user! See https://getcomposer.org/root for details
Continue as root/super user [yes]? yes
Creating a "laravel/laravel" project at "./example-app"
Installing laravel/laravel (v9.5.1)
- Downloading laravel/laravel (v9.5.1)
- Installing laravel/laravel (v9.5.1): Extracting archive
Created project in /Applications/XAMPP/xamppfiles/htdocs/example-app
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 107 installs, 0 updates, 0 removals
- Locking brick/math (0.10.2)
- Locking dflydev/dot-access-data (v3.0.2)
- Locking doctrine/deprecations (v1.0.0)
- Locking doctrine/inflexor (2.0.6)
- Locking doctrine/instantiator (2.0.0)
- Locking doctrine/lexer (2.1.0)
- Locking dragonmantank/cron-expression (v3.3.2)
- Locking egulias/email-validator (3.2.5)
- Locking fakerphp/faker (v1.21.0)
- Locking filp/whoops (2.14.6)
- Locking fruitcake/php-cors (v1.2.0)
- Locking graham-campbell/result-type (v1.1.0)
- Locking guzzlehttp/guzzle (7.5.0)
- Locking guzzlehttp/promises (1.5.2)
- Locking guzzlehttp/psr7 (2.4.3)
- Locking hamcrest/hamcrest-php (v2.0.1)
```

Cuidado con XAMPP y PHP la versión de PHP es la 8.2, y eso hace que no sea compatible con Laravel 12.X y 13.X que necesitan la versión 8.3 de PHP como mínimo.

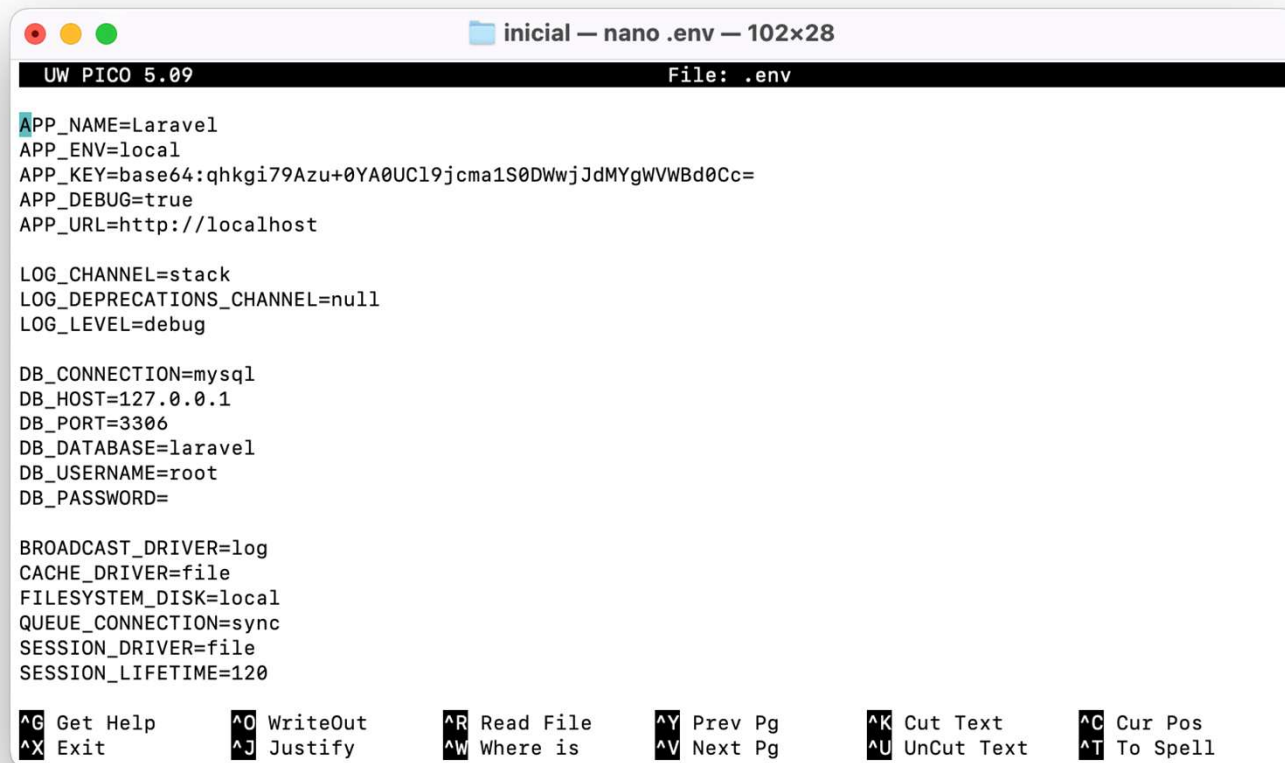
Instalación en local

Una vez dentro del proyecto nos encontraremos con esta estructura:

```
htdocs — bash — 102x28
sh-3.2# ls -la
total 656
drwxr-xr-x 25 root admin 800 24 ene 15:26 .
drwxr-xr-x 15 root admin 480 24 ene 15:26 ..
-rw-r--r-- 1 root admin 258 11 ene 16:50 .editorconfig
-rw-r--r-- 1 root admin 1120 24 ene 15:26 .env
-rw-r--r-- 1 root admin 1069 11 ene 16:50 .env.example
-rw-r--r-- 1 root admin 179 11 ene 16:50 .gitattributes
-rw-r--r-- 1 root admin 227 11 ene 16:50 .gitignore
-rw-r--r-- 1 root admin 4158 11 ene 16:50 README.md
drwxr-xr-x 7 root admin 224 11 ene 16:50 app
-rwxr-xr-x 1 root admin 1686 11 ene 16:50 artisan
drwxr-xr-x 4 root admin 128 11 ene 16:50 bootstrap
-rw-r--r-- 1 root admin 1817 11 ene 16:50 composer.json
-rw-r--r-- 1 root admin 286057 24 ene 15:26 composer.lock
drwxr-xr-x 17 root admin 544 11 ene 16:50 config
drwxr-xr-x 6 root admin 192 11 ene 16:50 database
drwxr-xr-x 3 root admin 96 11 ene 16:50 lang
-rw-r--r-- 1 root admin 286 11 ene 16:50 package.json
-rw-r--r-- 1 root admin 1175 11 ene 16:50 phpunit.xml
drwxr-xr-x 6 root admin 192 11 ene 16:50 public
drwxr-xr-x 5 root admin 160 11 ene 16:50 resources
drwxr-xr-x 6 root admin 192 11 ene 16:50 routes
drwxr-xr-x 5 root admin 160 11 ene 16:50 storage
drwxr-xr-x 6 root admin 192 11 ene 16:50 tests
drwxr-xr-x 40 root admin 1280 24 ene 15:26 vendor
-rw-r--r-- 1 root admin 263 11 ene 16:50 vite.config.js
sh-3.2#
```


Instalación en local

Para configurar nuestra app en el entorno local tenemos que editar .env



```
UW PICO 5.09                               File: .env

APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:qhkgi79Azu+0YA0UC19jcma1S0DWwjJdMYgVWVBd0Cc=
APP_DEBUG=true
APP_URL=http://localhost

LOG_CHANNEL=stack
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug

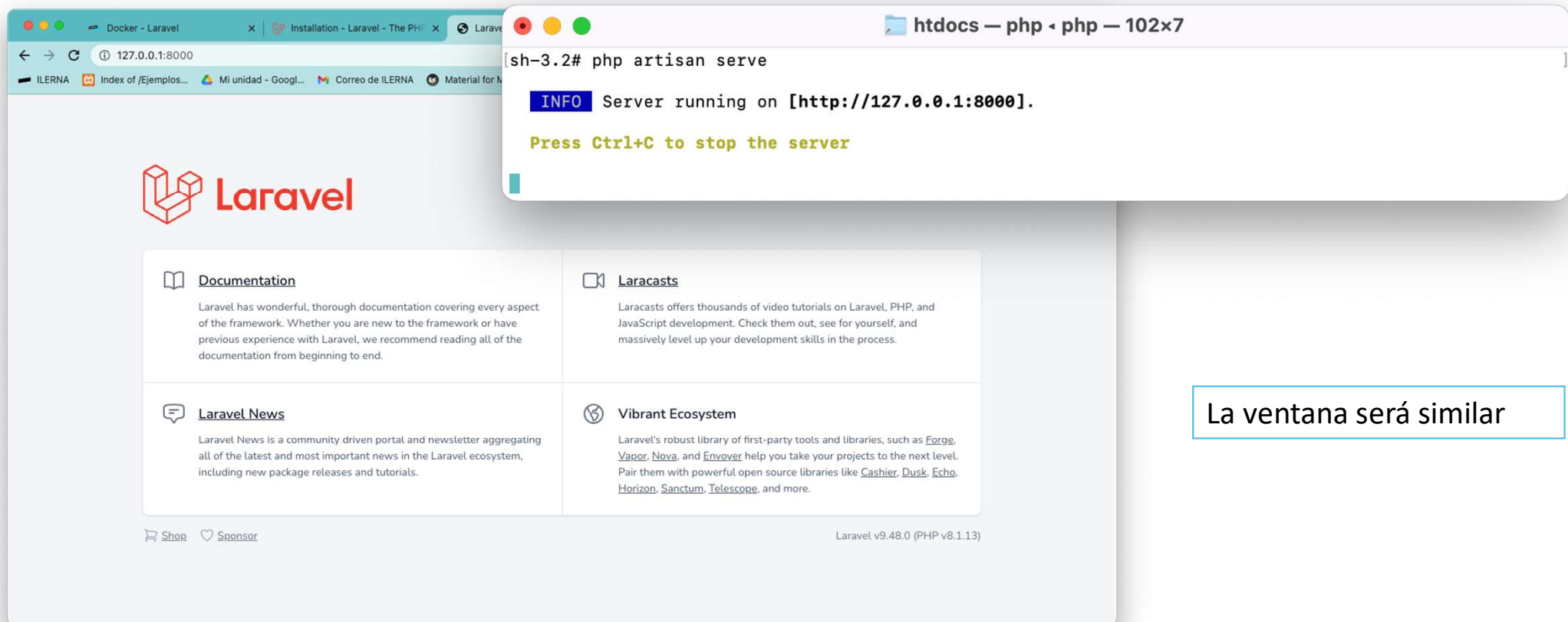
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=

BROADCAST_DRIVER=log
CACHE_DRIVER=file
FILESYSTEM_DISK=local
QUEUE_CONNECTION=sync
SESSION_DRIVER=file
SESSION_LIFETIME=120

^G Get Help      ^O WriteOut      ^R Read File     ^Y Prev Pg      ^K Cut Text      ^C Cur Pos
^X Exit          ^J Justify       ^W Where is     ^V Next Pg     ^U UnCut Text    ^T To Spell
```


Instalación en local

Para lanzar el servidor podemos usar:



The image shows a web browser window displaying the Laravel homepage. The browser's address bar shows the URL `127.0.0.1:8000`. The page features the Laravel logo and several links: [Documentation](#), [Laracasts](#), [Laravel News](#), and [Vibrant Ecosystem](#). At the bottom, there are links for [Shop](#) and [Sponsor](#), and the version information "Laravel v9.48.0 (PHP v8.1.13)".

Overlaid on the browser window is a terminal window titled "htdocs — php — 102x7". The terminal shows the command `[sh-3.2# php artisan serve]` and the output: `INFO Server running on [http://127.0.0.1:8000].` Below this, it says `Press Ctrl+C to stop the server`.

La ventana será similar

Instalación en local

Ahora entramos en la carpeta database/migrations, vemos que se han creado unos archivos.

```
migrations — -zsh — 102x9

/Applications/XAMPP/htdocs — php ◀ php

Last login: Tue Jan 24 14:49:42 on ttys000
olga@MacBook-Pro-de-olga htdocs % cd example-app
olga@MacBook-Pro-de-olga example-app % cd database/migrations
olga@MacBook-Pro-de-olga migrations % ls
2014_10_12_000000_create_users_table.php
2014_10_12_100000_create_password_resets_table.php
2019_08_19_000000_create_failed_jobs_table.php
2019_12_14_000001_create_personal_access_tokens_table.php
olga@MacBook-Pro-de-olga migrations %
```

El *unique* impide que se le añadan emails iguales a dos usuarios diferentes. Esto incluye que no se le puede dar un dato vacío a ambos.

rememberToken es un código modificado al entrar y salir, para evitar accesos indeseados.

Timestamps permite a Laravel identificar el orden de las migraciones.

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });
    }
}
```

Instalación en local

```
example-app — -zsh — 67x31
...ns/XAMPP/htdocs — php • php
...docs/example-app — -zsh +
[olga@MacBook-Pro-de-olga migrations % php artisan migrate
Could not open input file: artisan
[olga@MacBook-Pro-de-olga migrations % cd ..
[olga@MacBook-Pro-de-olga database % cd ..
[olga@MacBook-Pro-de-olga example-app % php artisan migrate

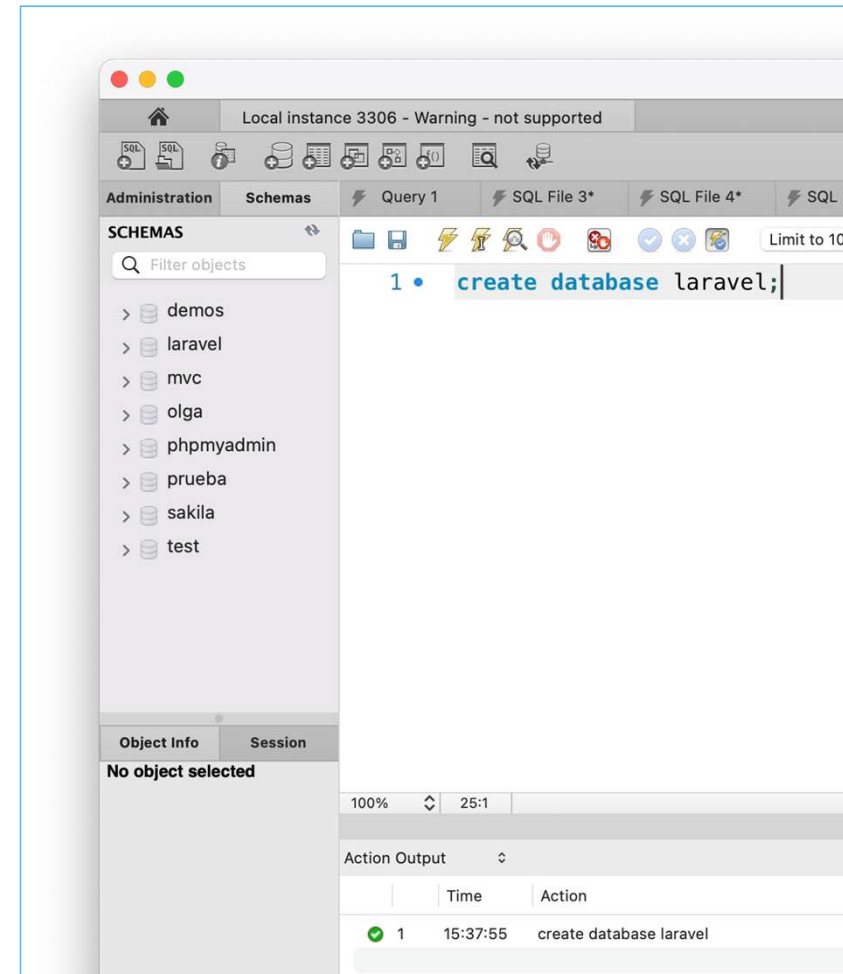
INFO Preparing database.

Creating migration table ..... 39ms DONE

INFO Running migrations.

2014_10_12_000000_create_users_table ..... 41ms DONE
2014_10_12_100000_create_password_resets_table ..... 42ms DONE
2019_08_19_000000_create_failed_jobs_table ..... 30ms DONE
2019_12_14_000001_create_personal_access_tokens_table 35ms DONE

olga@MacBook-Pro-de-olga example-app %
```



Migraciones



Con las migraciones vamos a gestionar la base de datos de nuestro sitio web; tanto crear nuevas BBDD como editarlas desde Laravel.

Las migraciones de un sitio hecho con Laravel se alojan en la ruta database/migrations y tienen extensión .php.

Si te fijas, nada más instalar Laravel, se han creado varios archivos de migraciones que podemos usar, pero de momento los dejamos ahí y NO LOS BORRAMOS porque son muy útiles.

Archivos .env

Es de uso común trabajar con archivos de entorno llamados también archivos .env. Normalmente, en un proyecto real puedes encontrarte con varios archivos de este tipo en función del despliegue que se quiera hacer; como por ejemplo:

- test.env.config. para desplegar en entorno seguro de pruebas
- release.env.config. para desplegar cambios de versión que se envía a los beta testers
- production.env.config. para desplegar el código ya probado al resto del mundo.

En nuestro caso, como no vamos a desplegar nada, sólo vamos a usar un único archivo .env que vimos cómo utilizar en la instalación. En este archivo debemos configurar los datos de nuestro servidor MySQL y rellenarlo con la información correspondiente a nuestra base de datos **ya creada**.

Migraciones

Una vez tengamos esto, lo que nos queda es ejecutar el comando de las migraciones a través del CLI `artisan`: `php artisan migrate`

Si todo ha salido bien obtendremos el siguiente resultado donde podremos observar que todas las migraciones se han insertado correctamente en la base de datos.

```
[olga@MacBook-Pro-de-olga example-app % php artisan migrate]
[INFO] Preparing database.

Creating migration table ..... 39ms DONE

[INFO] Running migrations.

2014_10_12_000000_create_users_table ..... 41ms DONE
2014_10_12_100000_create_password_resets_table ..... 42ms DONE
2019_08_19_000000_create_failed_jobs_table ..... 30ms DONE
2019_12_14_000001_create_personal_access_tokens_table 35ms DONE
```

Si nos vamos al cliente que utilizemos para manejar la base de datos (Workbench, por ejemplo) veremos que en nuestra base de datos se han creado todas las tablas de la migración que hemos ejecutado y además una tabla que se llama migrations.

La tabla migrations es simplemente un registro de todas las migraciones llevadas a cabo. Además, podemos hacer un rollback en caso de que queramos reestablecer nuestra BBDD

`php artisan migrate:rollback`

O si preferimos hacer un reset para dejarla limpia:

`php artisan migrate:reset`

Carpetas en Laravel

Al crear un nuevo proyecto con este framework, Laravel crea una serie de carpetas por defecto. Esta estructura de carpetas es la recomendada para utilizar Laravel.

Public: Esta es la carpeta más importante ya que es donde se ponen todos los archivos que el cliente va a mostrar al usuario cuando introduzcamos la URL de nuestro sitio web. Normalmente se carga el archivo `index.php` por defecto.

Routes: Otra de las carpetas que más vamos a usar a lo largo de este curso de Laravel. En ella se albergan todas las rutas (redirecciones web) de nuestro proyecto, pero más concretamente en el archivo `web.php`. Dada una ruta → se cargará una vista

Resources: Esta es nuestra carpeta de recursos donde guardaremos los siguientes archivos, que también, están separados por sus carpetas... como cada nombre indica:

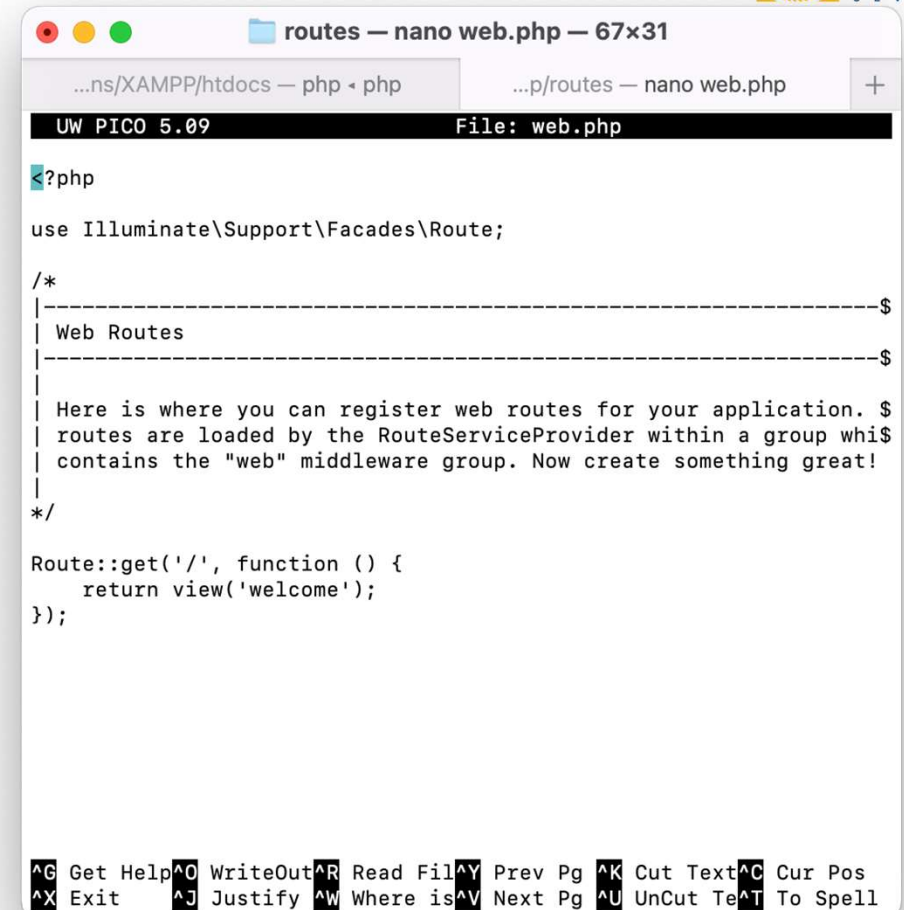
- **css** Archivos CSS
- **js** Archivos JS (JavaScript)
- **lang** Archivos relacionados con el idioma del sitio (variables & strings)
- **views** Archivos de nuestras vistas, lo que las rutas cargan

Rutas

Las rutas en Laravel (y en casi cualquier Framework) sirven para redireccionar al cliente (o navegador) a las vistas que nosotros queramos.

Estas rutas se configuran en el archivo `public/routes/web.php` donde se define la ruta que el usuario pone en la URL después del dominio y se retorna la vista que se quiere cargar al introducir dicha dirección en el navegador.

En el ejemplo de arriba vamos a cargar la vista llamada `welcome` que hace referencia a la vista `resources/views/welcome.blade.php`



```
routes — nano web.php — 67x31
...ns/XAMPP/htdocs — php < php  ...p/routes — nano web.php
UW PICO 5.09 File: web.php
?php
use Illuminate\Support\Facades\Route;

/*
|-----$
| Web Routes
|-----$
|
| Here is where you can register web routes for your application. $
| routes are loaded by the RouteServiceProvider within a group whi$
| contains the "web" middleware group. Now create something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

^G Get Help ^O WriteOut ^R Read Fil ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Te ^T To Spell
```


Rutas - alias

Es interesante darle un alias o un nombre a nuestras rutas para poder utilizar dichos alias en nuestras plantillas de Laravel que veremos más adelante.

Para ello, basta con utilizar la palabra `name` al final de la estructura de la ruta y darle un nombre que queramos; normalmente descriptivo y asociado a la vista que tiene que cargar el enrutador de Laravel.

```
<?php

Route::get('/users', function () {
    return view('users');
}) -> name('usuarios');
```

```
<?php

/* http://localhost/datos/ */

Route::view('datos', 'usuarios');
```

Después veremos que es muy útil ya que a la hora de refactorizar o hacer un cambio, si tenemos enlaces o menús de navegación que apuntan a esta ruta, sólo tendríamos que cambiar el parámetro dentro del `get()` y no tener que ir archivo por archivo.

Laravel nos proporciona una manera más cómoda a la hora de cargar una vista si no queremos parámetros ni condiciones. Tan sólo definiremos la siguiente línea que hace referencia la ruta `datos` en la URL y va a cargar el archivo `usuarios.php` de nuestra carpeta `views` como le hemos indicado en el segundo parámetro.

Rutas - parámetros

Ya hemos visto que con PHP podemos pasar parámetros a través de la URL, como si fueran variables, que las recuperábamos a través del método GET o POST.

Con Laravel también podemos introducir parámetros pero de una forma más vistosa y ordenada, de tal manera que sea visualmente más cómodo de recordar y de indexar por los motores de búsqueda como Google.

<http://localhost/cliente/324>

Para configurar este tipo de rutas en nuestro archivo de rutas `public/routes/web.php`, tenemos que considerar algunas cosas: ¿Qué pasa si no introducimos un id y sólo navegamos hasta `cliente/` ? ... Nos va a devolver un 404.

Para resolver esto, podemos definir una ruta por defecto en caso de que el id (o parámetro) no sea pasado. Para ello usaremos el símbolo `?` en nuestro nombre de ruta e inicializaremos la variable con el valor que queramos. También podremos agregar una expresión regular para obligar al parámetro a cumplir con ser un número.

```
<?php
```

```
Route::get('cliente/{id?}', function($id = 1) {  
    return 'Cliente con el id: ' . $id;  
}) -> where('id', '[0-9]+');
```

Olga M. Moreno Martín

Estructuras de control

Como en todo buen lenguaje de programación, en Laravel también tenemos estructuras de control.

En Blade (plantillas de Laravel) siempre que iniciemos un bloque de estructura de control DEBEMOS cerrarla:

- @foreach ~ @endforeach lo usamos para recorrer arrays
- @if ~ @endif para comprobar condiciones lógicas
- @switch ~ @endswitch en función del valor de una variable ejecutar un código
- @case define la casuística del switch
- @break rompe la ejecución del código en curso
- @default si ninguna casuística se cumple

Acordaros que podemos pasar variables a través de las rutas como si fueran parámetros. Pero en este caso, vamos a ver otra directiva más; el uso de @compact.

```
<?php

$equipo = ['María', 'Alfredo', 'William', 'Verónica'];

@foreach ($equipo as $nombre)
    <p> {{ $nombre }} </p>
@endforeach
```

```
<?php

// Uso de @compact
$equipo = ['María', 'Alfredo', 'William', 'Verónica'];

// Route::view('nosotros', ['equipo' => 'equipo']);
Route::view('nosotros', @compact('equipo'));
```

Plantillas o templates

A través de las plantillas de Laravel vamos a escribir menos código PHP y vamos a tener nuestros archivos mejor organizados.

Blade es el sistema de plantillas que trae Laravel, por eso los archivos de plantillas que guardamos en el directorio de views llevan la extensión **blade.php**.

De esta manera sabemos inmediatamente que se trata de una plantilla de Laravel y que forma parte de una vista que se mostrará en el navegador.

Directivas

Laravel tiene un gran número de directivas que podemos utilizar para ahorrarnos mucho código repetitivo entre otras funciones.

Digamos que las directivas son pequeñas funciones ya escritas que aceptan parámetros y que cada una de ellas hace una función diferente dentro de Laravel.

@yield Define el contenido dinámico que se va a cargar. Se usa conjuntamente con **@section**. Es decir, que lo que declaramos con **@yield** luego lo “rellenamos” con **@section**.

@section y **@endsection** bloque de código dinámico

@extends importa el contenido de una plantilla ya creada para poder utilizarla

Ejercicio 1

1. Crea un sitio web con Laravel que contenga el título "Bienvenidos a Laravel, DAW", un texto de bienvenida (puede ser un poco de Lorem Ipsum) y a continuación un menú de navegación con sus correspondientes alias y los siguientes enlaces:

- a) Inicio enlace a la página principal donde se visualizará el texto de Lorem Ipsum además de los elementos estáticos (Título y menú de navegación).
- b) ¿Quiénes somos? enlace que vaya a la página "nosotros" y muestre, además de los elementos estáticos de todo el sitio, un h2 que diga "Estás en la sección sobre nosotros".
- c) Proyecto enlace que cargue una vista con el siguiente texto "Estás en el proyecto número: X" donde X es un número entero que podamos introducirlo en la propia ruta. Si no se mete ningún número en la ruta, por defecto tiene que ser 1; por ejemplo

<http://localhost:8000/proyecto/210937>

Recuerda que el título y el menú de navegación han de aparecer en todas las vistas que cargues. (Usa las plantillas)

Modelos

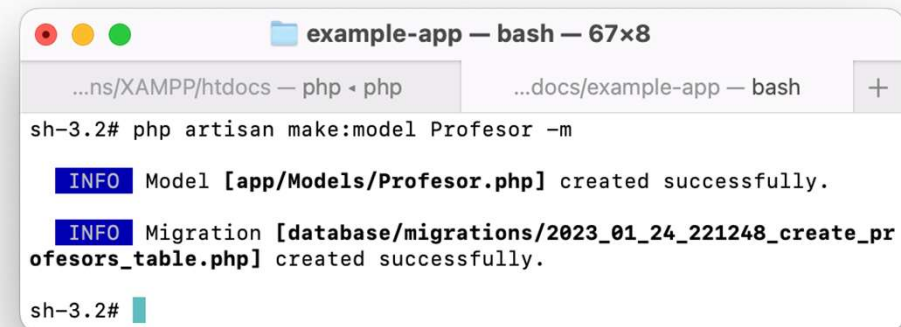
Gracias a Eloquent y su integración con Laravel, podremos crear modelos de datos de una manera automatizada a través de artisan

Ahora que ya sabemos manejar las migraciones es hora de crear nuestras propias migraciones pero a través de Eloquent.

A través de la instrucción `make:model` creamos un nuevo modelo de datos, a continuación ponemos el nombre **siempre empezando en Mayúscula y en SINGULAR** y pasamos el parámetro relacionado con las migraciones `-m`.

```
php artisan make:model Profesor -m
```

Si todo ha salido bien, veremos en nuestro directorio de migraciones `database/migrations` un nuevo archivo que se llama `2022_01_07_81237_create_profesor_table.php`.



```
example-app — bash — 67x8
...ns/XAMPP/htdocs — php < php
...docs/example-app — bash
sh-3.2# php artisan make:model Profesor -m

[INFO] Model [app/Models/Profesor.php] created successfully.
[INFO] Migration [database/migrations/2023_01_24_221248_create_profesors_table.php] created successfully.

sh-3.2#
```

Modelos



En esta Schema podemos poner todas las tablas que nosotros queramos y establecer el tipo de dato para cada una de ellas, así como el nombre.

Una vez que estamos satisfechos con nuestro esquema debemos volver a ejecutar el código de migración a través de artisan para que se introduzca esta nueva información en la base de datos.

Para ver todos los tipos de datos que maneja Eloquent puedes visitar la documentación oficial.

<https://laravel.com/docs/11.x/migrations#available-column-types>

Modelos – Relaciones con Eloquent

A través de Eloquent vamos a poder gestionar las relaciones entre nuestras tablas de la base de datos de una manera muy sencilla y sin sentencias SQL.

Uno a uno (1 a 1)

Para crear este tipo de relaciones en Eloquent y Laravel, debemos tener creadas las tablas que vayamos a relacionar y establecer la relación entre ellas a través del método `hasOne`.

Supongamos que tenemos una tabla usuario que está relacionada con la tabla teléfono.

En este caso, Eloquent asume que en Usuario existe la FK `user_id` pero ¿qué pasa si tenemos otro nombre? Pues se lo pasamos como parámetro.

Si la PK no se llama id, también debemos pasarla por parámetro.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Get the phone associated with the user.
     */
    public function phone()
    {
        return $this->hasOne(Phone::class);
    }
}

return $this->hasOne(Phone::class, 'foreign_key', 'local_key');
```

Modelos – Relaciones con Eloquent

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Phone extends Model
{
    /**
     * Get the user that owns the phone.
     */
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

return \$this->belongsTo(User::class, 'foreign_key', 'owner_key');

Uno a Uno INVERSA

Ahora que podemos acceder al modelo teléfono desde el modelo usuario, vamos a ver cómo hacerlo de manera inversa, es decir, cómo acceder desde el modelo usuario desde el modelo teléfono gracias al método belongsTo().

Modelos – Relaciones con Eloquent

Uno a Muchos (1 a M)

En este caso, las relaciones de 1 a muchos podemos decir que en una entrada de un blog, o en un post de Facebook, hay muchos comentarios relacionados a esa misma publicación.

Para empezar, ya sabemos que debemos crear el modelo y en este caso usaremos el método `hasMany()` para obtener los datos relacionados con ese post o entrada en el blog.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    /**
     * Get the comments for the blog post.
     */
    public function comments()
    {
        return $this->hasMany(Comment::class);
    }
}
```

Modelos – Relaciones con Eloquent

Uno a Muchos (1 a M)

Cuidado con las claves ajenas, que aquí pasa lo mismo...

Eloquent establece por defecto el sufijo `_id` por lo tanto, en este ejemplo buscaría por `post_id`. Si no queremos eso o nuestra clave ajena tiene otro nombre, se lo pasamos por parámetro en el método `hasMany` como hacíamos más arriba.

Ahora, al haber más de un dato, necesitamos iterar, por tanto debemos crear un bucle para poder sacar cada dato.

Además, como todas las relaciones son sentencias SQL, podemos anidar varios filtros en función de lo que queramos sacar.

```
return $this->hasMany(Comment::class, 'foreign_key');  
  
return $this->hasMany(Comment::class, 'foreign_key', 'local_key');
```

```
use App\Models\Post;  
  
$comments = Post::find(1)->comments;  
  
foreach ($comments as $comment) {  
    //  
}
```

```
$comment = Post::find(1)->comments()  
    ->where('title', 'foo')  
    ->first();
```

Modelos – Relaciones con Eloquent

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    /**
     * Get the post that owns the comment.
     */
    public function post()
    {
        return $this->belongsTo(Post::class);
    }
}
```

Uno a Muchos INVERSA

Ahora que podemos acceder a todos los comentarios de una publicación, definamos una relación para permitir que un comentario acceda a su publicación principal.

Y ahora, a través de la propiedad de relación dinámica...

Pasaría lo mismo con el nombre de la clave ajena, si no se llama de la misma manera que Eloquent establece con el sufijo _id podemos pasarle como parámetro el nombre de la clave donde debe buscar.

```
use App\Models\Comment;

$comment = Comment::find(1);

return $comment->post->title;
```

Modelos – Relaciones con Eloquent

Muchos a Muchos (N a M)

Este tipo de relaciones son las más complicadas ya que, en un Blog del estilo Wordpress por ejemplo, un usuario puede tener muchos roles (lector, autor, administrador) pero un rol pueden tenerlo varios usuarios, es decir, puede haber muchos usuarios administradores, otros lectores y demás.

Para realizar este tipo de relaciones necesitaríamos 3 tablas diferentes.

Lo primero de todo, vamos a crear las tablas con sus modelos a excepción de la tabla pivot `role_user` que sólo crearemos la tabla, sin su modelo.

↓
`php artisan make:migration create_role_user_table --create=role_user`

```
users
  id - integer
  name - string

roles
  id - integer
  name - string

role_user
  user_id - integer
  role_id - integer
```

```
$table->id();
$table->unsignedInteger('user_id');
$table->unsignedInteger('role_id');
$table->timestamps();
```

Modelos – Relaciones con Eloquent

Muchos a Muchos (N a M)

Recordad User.php ya existe.

Y ahora, a través de la propiedad de relación dinámica...

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The roles that belong to the user.
     */
    public function roles()
    {
        return $this->belongsToMany(Role::class);
    }
}
```

```
return $this->belongsToMany(Role::class)->withTimestamps();
```

```
return $this->belongsToMany(Role::class, 'role_user', 'user_id', 'role_id');
```

```
use App\Models\User;

$user = User::find(1);

foreach ($user->roles as $role) {
    //
}
```

```
$roles = User::find(1)->roles()->orderBy('name')->get();
```


Modelos – Relaciones con Eloquent

Muchos a Muchos INVERSA

Para definir el "inverso" de una relación de muchos a muchos, debemos establecer un método en el modelo relacionado que también devuelva el resultado del método `belongsToMany`. Según el ejemplo que estamos siguiendo...

Si usamos la declaración (ACONSEJABLE) tendremos el atributo `pivot` disponible con los timestamps, recordad, hay veces que queremos guardar más atributos en la tabla pivote.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Role extends Model
{
    /**
     * The users that belong to the role.
     */
    public function users()
    {
        return $this->belongsToMany(User::class);
    }
}
```

```
use App\Models\User;

$user = User::find(1);

foreach ($user->roles as $role) {
    echo $role->pivot->created_at;
}
```

Ejercicio 2



2. Vamos a hacer un ejemplo con una APP que gestione alumnos y asignaturas, de tal manera que MUCHOS ALUMNOS pueden cursar MUCHAS ASIGNATURAS

¿Qué necesitamos para este ejemplo?

- 3 migraciones para crear las tablas
- Alumnos /// Materias /// AlumnoMateria
- Modificar los archivos de las migraciones `create_alumnos_table` y `create_materias_table`.
- Ejecutar las Migraciones
- 2 modelos para Alumnos y Materias
- Método dentro de Alumno para crear la relación Alumno -> Materia y en materia para saber que alumnos las tienen (la inversa).
- Rellenar la base de datos

Controladores

Los controladores son el lugar perfecto para definir la lógica de negocio de nuestra aplicación o sitio web.

Hace de intermediario entre la vista (lo que vemos con nuestro navegador o cliente) y el servidor donde la app está alojada.

Por defecto, los controladores se guardan en una carpeta específica situada en `app/Http/Controllers` y tienen extensión `.php`. Se llaman igual que el modelo al que referencian pero con **nombre en PLURAL**, con la **primera en MAYÚSCULAS**.

Para crear un controlador nuevo debemos hacer uso de nuestro querido artisan donde le diremos que cree un controlador con el nombre que nosotros queramos.

Abrimos la consola y nos situamos en la raíz de nuestro proyecto, como siempre:

```
php artisan make:controller PagesController
```

Si todo ha salido bien, recibiremos un mensaje por consola con que todo ha ido bien y podremos comprobar que, efectivamente se ha creado el archivo `PagesController.php` con una estructura básica de controlador, dentro de la carpeta `Controllers` que hemos descrito anteriormente.

Ahora podemos modificar nuestro archivo de rutas `web.php` para dejarlo limpio de lógica y trasladar ésta a nuestro nuevo controlador.

La idea de esto es dejar el archivo `web.php` tan limpio como podamos para que, de un vistazo, se entienda todo perfectamente.

RECUERDA que sólo movemos la lógica, mientras que las cláusulas como `where` y `name` las seguimos dejando en el archivo de rutas `web.php`

Miniejemplo:

```
<body>
  <!-- CONTENIDO ESTÁTICO PARA TODAS LAS SECCIONES -->
  <h1>Bienvenid@s a Laravel</h1>
  <hr>

  <!-- MENÚ -->
  <nav>
    <a href={{ route('noticias') }}>Blog</a> |
    <a href={{ route('galeria') }}>Fotos</a>
  </nav>

  <!-- CONTENIDO DINÁMICO EN FUNCIÓN DE LA SECCIÓN QUE SE VISITA -->
  <header>
    @yield('apartado')
  </header>
</body>
```

```
<?php
// blog.blade.php

@extends('plantilla')

@section('apartado')
  <h2>Estás en BLOG</h2>
@endsection
```

```
<?php
// fotos.blade.html

@extends('plantilla')

@section('apartado')
  <h2>Estás en FOTOS</h2>
@endsection
```

```
<?php

// web.php

Route::view('blog', 'blog') -> name('noticias');
Route::view('fotos', 'fotos') -> name('galeria');
```

```
// Uso de @compact
$equipo = ['María', 'Alfredo', 'William', 'Verónica'];

// Route::view('nosotros', ['equipo' => 'equipo']);
Route::view('nosotros', @compact('equipo'));
```

```
$equipo = ['María', 'Alfredo', 'William', 'Verónica'];

@foreach ($equipo as $nombre)
  <p> {{ $nombre }} </p>
@endforeach
```

Falta el
nosotros.blade.php

Miniejemplo:



```
<?php

// PagesController.php

namespace App\Http\Controllers;

class PagesController extends Controller
{
    public function inicio() { return view('welcome'); }

    public function datos() {
        return view('usuarios', ['id' => 56]);
    }

    public function cliente($id = 1) {
        return ('Cliente con el id: ' . $id);
    }

    public function nosotros($nombre = null) {
        $equipo = [
            'Paco',
            'Enrique',
            'Maria',
            'Veronica'
        ];

        return view('nosotros', @compact('equipo', 'nombre'));
    }
}
```

```
<?php

// web.php (v2.0) Refactorizado

use App\Http\Controllers\PagesController;
use Illuminate\Support\Facades\Route;

Route::get('/', [ PagesController::class, 'inicio' ]);
Route::get('datos', [ PagesController::class, 'datos' ]);
Route::get('cliente/{id?}', [ PagesController::class, 'cliente' ]) -> where('id', '[0-9]+');
Route::get('nosotros/{nosotros?}', [ PagesController::class, 'nosotros' ]) -> name('nosotros');
```

Ejercicio 3



Partiendo del ejercicio 2:

- a) Crear el controlador para la vista
- b) Crear la ruta de nuestra vista
- c) Crear la vista con los datos

Formularios - Previa

Para trabajar con formularios, vamos a usar un modelo muy sencillo.
Le vamos a llamar Nota, y va a tener los atributos nombre y descripción.

```
php artisan make:model Nota -m  
php artisan make:controller NotasController
```

En la migration:

```
<?php  
Schema::create('notas', function (Blueprint $table) {  
    $table->id();  
    $table->timestamps();  
    $table->string('nombre');  
    $table->text('descripcion');  
});
```

Formularios - Previa

Una vez creado el controlador vamos a routes/web.php:

```
Route::get('notas', [ NotasController::class, 'notas' ]);
```

Luego editamos el controlador:

```
public function notas() {  
    $notas = Nota::all(); // Nos saca todas las notas de la BBDD  
    return view('notas', @compact('notas'));  
}
```

Ahora, creamos la vista con la tabla que pinte los datos de la variable 'notas' de la ruta.

Formularios - Previa

```
// notas.blade.php
```

```
<h1>Notas desde base de datos</h1>
```

```
<table border="1">
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Nombre</th>
```

```
      <th>Descripción</th>
```

```
    </tr>
```

```
  </thead>
```

```
  @foreach ($notas as $nota)
```

```
    <tr>
```

```
      <td>{{$nota -> nombre}}</td>
```

```
      <td>{{$nota -> descripcion}}</td>
```

```
    </tr>
```

```
  @endforeach
```

```
</table>
```

Formularios - Previa

Hay que fijarse bien en los nombres de las columnas que tienen nuestras bases de datos, es justo lo que va después de -> y siempre rodeado por los símbolos {{ }} ya que estamos en un archivo de plantilla.

¿Qué pasaría si sólo queremos acceder a un único elemento? como si hiciésemos un
SELECT * from usuarios where id = 1

Para eso, tenemos una instrucción específica en Eloquent que nos soluciona el problema. En este caso usaremos la instrucción findOrFail y como buenos usuarios de Laravel, lo utilizaremos dentro del controlador NotasController.php.

```
public function detalle($id) {  
    $nota = Nota::findOrFail($id);  
    return view('notas.detalle', @compact('nota'));  
}
```

Hay que acordarse que debemos configurar la ruta en nuestro archivo de rutas.

```
Route::get('notas/{id?}', [ NotasController::class, 'detalle' ]) -> name('notas.detalle');
```

Y por último, debemos crear la plantilla, pero como es un archivo de detalle o que está relacionado con otra plantilla ya creada, podemos crear una carpeta con el nombre de la plantilla y dentro, el archivo de plantilla en cuestión.

Formularios - Previa



```
//detalle.blade.php
```

```
<h1>Detalle de la nota</h1>
```

```
<h3>ID: {{ $nota -> id }}</h3>
```

```
<h3>Nombre: {{ $nota -> nombre }}</h3>
```

```
<h3>Descripción: {{ $nota -> descripcion }}</h3>
```

Formularios

Ahora que ya sabemos cómo cargar de una base de datos, vamos a ver cómo insertarlos con Laravel y sin escribir ni una sola línea de SQL.

Gracias al método `save()` de Laravel podremos guardar datos que provengan de un formulario desde nuestras plantillas. Para ello, lo primero que necesitamos lo siguiente:

- Formulario HTML que recoja los datos que el usuario introduce
- Una ruta que sea la encargada de recibir los datos del formulario
- Método POST para enviar los datos al servidor
- Un método en nuestro controlador que procese los datos y los guarde a través de `save()`
- La cláusula de seguridad `@csrf` para evitar ataques desde otros sitio.

Así pues, empecemos por el formulario:

```
<form action="{{ route('notas.crear') }}" method="POST">
    @csrf {{-- Cláusula para obtener un token de formulario al enviarlo --}}
    <input type="text" name="nombre" placeholder="Nombre de la nota" class="form-control mb-2" autofocus>
    <input type="text" name="descripcion" placeholder="Descripción de la nota" class="form-control mb-2">
    <button class="btn btn-primary btn-block" type="submit">
        Crear nueva nota
    </button></form>
```

Formularios

Como vemos, creamos 2 inputs relacionados con nuestras columnas dentro de la tabla, en este caso nombre y descripción

El action del formulario debe apuntar a una nueva ruta que vayamos a crear y donde enviemos los datos mediante POST.

Ahora crearemos la ruta en nuestro archivo de rutas web.php

```
Route::post('notas', [ NotasController::class, 'crear' ]) -> name('notas.crear');
```

Si nos fijamos, ya no estamos haciendo uso del get sino del método post y como son métodos diferentes, podemos nombrar la ruta de la misma manera que en get ya que no habrá conflicto.

Por otro lado, necesitamos invocar nuestro NotasController y decirle que vamos a utilizar el método crear que bueno, todavía no existe pero que vamos a crear a continuación. No olvidemos crear un alias para poder vincularlo al action del formulario **SUPER IMPORTANTE**.

Para terminar, editaremos nuestro archivo NotasController.php para que el controlador que estamos usando tenga el método que hemos nombrado previamente.

Formularios

```
// estamos en NotasController.php  
public function crear(Request $request) {  
    $notaNueva = new Nota;  
    $notaNueva -> nombre = $request -> nombre;  
    $notaNueva -> descripcion = $request -> descripcion;  
    $notaNueva -> save();  
    return back() -> with('mensaje', 'Nota agregada exitosamente');  
}
```

Nuestro método crear va a recibir un objeto llamado \$request (el cual podemos cambiarle el nombre perfectamente) de tipo Request por lo que **DEBEMOS IMPORTAR Request** para poder utilizar dicha clase de Laravel.

```
use Illuminate\Http\Request;
```

Formularios

Dentro del método crear que acabamos de escribir, debemos hacer uso del modelo Nota que ya creamos en otros ejemplos e instanciarla a través de una variable, en nuestro caso \$notaNueva.

De esta manera, podremos acceder a los métodos y propiedades de la misma, por eso podemos utilizar nombre y descripción.

Así mismo, la variable \$request está formada por los mismos nombres de input que hemos puesto en el formulario de la plantilla, si os acordáis eran name="nombre" y name="descripcion".

Finalmente usamos el método save() para guardar estos nuevos datos en la base de datos y retornamos a la página del formulario con el método back() añadiendo un mensaje con with().

Pero ¿dónde va a salir este mensaje? -- lo tenemos que declarar en nuestra plantilla

```
@if (session('mensaje'))  
    <div class="mensaje-nota-creada">  
        {{ session('mensaje') }}  
    </div>  
@endif
```

Validaciones

Laravel nos proporciona herramientas para poder validar los datos que el usuario introduce en los campos del formulario.

Además de poder hacerlo con la etiqueta `required` de HTML5, debemos validar los datos a través del Framework.

Para ello, necesitamos modificar varios elementos:

En primer lugar, nuestro archivo controller

En segundo lugar, nuestra plantilla que carga el formulario

Empecemos con el controlador. A través del método `validate()` le decimos a Eloquent qué campos son requeridos para poder enviar el formulario. Utilizaremos para ello un array asociativo con el nombre del input y la palabra reservada `required`

```
$request -> validate([ 'nombre' => 'required', 'descripcion' => 'required' ]);
```

Seguidamente nos moveremos a la vista donde esté el formulario y a través de la directiva `@error` crearemos un bloque html con nuestro mensaje de error por cada uno de los inputs requeridos.

```
@error('nombre') <div class="alert alert-danger"> No olvides rellenar el nombre  
</div> @enderror
```


Validaciones

Pero ¿qué pasa cuando ha habido un error y nos muestra el mensaje que hemos escrito? Si te fijas, los campos que habías rellenado perderán la información, pero con Laravel podemos persistirlos sin hacer que el usuario vuelva a introducirlos.

Para poder persistir los datos una vez enviados pero con algún error de campo requerido, utilizaremos la directiva `old()` como value del input dentro de nuestro formulario y le pasaremos el nombre del input declarado en la etiqueta name.

```
// estamos en notas.blade.php
```

```
<input  
  type="text"  
  name="nombre"  
  value="{{ old('nombre') }}"  
  class="form-control mb-2"  
  placeholder="Nombre de la nota"  
  autofocus >
```

Editando registros

Después de tener campos en la base de datos, lo interesante sería poder editarlos a través de un formulario.

Laravel nos proporciona las herramientas necesarias para ello; veamos pues lo que necesitamos para poder realizar el cambio a través de la directiva put().

Para poder hacer el cambio de registros necesitamos lo siguiente:

Un enlace para redirigir a la página de editar, pasando el id del elemento en cuestión

Una nueva ruta que apunte a nuestra plantilla de editar

Una plantilla para poder editar con un formulario que reciba los datos a editar

Una nueva función dentro de nuestro controlador para poder manejar los datos ya introducidos

Una nueva ruta que utilice el método put() para poder actualizar los datos

Un nuevo método dentro de nuestro controlador para actualizar los datos nuevos introducidos

```
<a href="{{ route('notas.editar', $nota) }}" class="btn btn-warning btn-sm"> Editar  
</a>
```

Editando registros

Ahora creamos la ruta:

```
Route::get('editar/{id}', [ NotasController::class, 'editar' ]) -> name('notas.editar');  
Route::put('editar/{id}', [ NotasController::class, 'actualizar' ]) ->  
name('notas.actualizar');
```

Necesitamos una nueva vista:

resources/views/notas/editar.blade.php

```
<h2>Editando la nota {{ $nota -> id }}</h2> @if (session('mensaje')) <div class="alert  
alert-success">{{ session('mensaje')}}</div> @endif <form action="{{  
route('notas.actualizar', $nota -> id) }}" method="POST">@method('PUT') {{--  
Necesitamos cambiar al método PUT para editar --}} @csrf {{-- Cláusula para  
obtener un token de formulario al enviarlo --}} @error('nombre') <div class="alert  
alert-danger"> El nombre es obligatorio </div> @enderror @error('descripcion') <div  
class="alert alert-danger"> La descripción es obligatoria </div> @enderror <input  
type="text" name="nombre" class="form-control mb-2" value="{{ $nota -> nombre }}"  
placeholder="Nombre de la nota" autofocus > <input type="text" name="descripcion"  
placeholder="Descripción de la nota" class="form-control mb-2" value="{{ $nota ->  
descripcion }}" > <button class="btn btn-primary btn-block" type="submit">Guardar  
cambios</button> </form>
```

Editando registros

Y por último, modificamos nuestro Controlador:

```
public function editar($id) {  
    $nota = Nota::findOrFail($id);  
    return view('notas.editar', compact('nota'));  
}  
  
public function actualizar(Request $request, $id) {  
    $request -> validate([  
        'nombre' => 'required',  
        'descripcion' => 'required'  
    ]);  
    $notaActualizar = Nota::findOrFail($id);  
    $notaActualizar -> nombre = $request -> nombre;  
    $notaActualizar -> descripcion = $request -> descripcion;  
    $notaActualizar -> save();  
    return back() -> with('mensaje', 'Nota actualizada');  
}
```

Eliminando registros

A la hora de eliminar un registro nuevo, no necesitamos crear una plantilla nueva ya que podemos mandar la instrucción directamente a través de otro formulario.

Por lo tanto, para eliminar un registro de la base de datos utilizaremos lo siguiente.

Un formulario básico con un único botón de eliminar

Usaremos el método DELETE para sobrescribir el método del formulario HTML

Una ruta nueva para controlar el action de este nuevo formulario

Un nuevo método dentro de nuestro Controlador que lleve la lógica para borrar el registro

Vamos a ver cómo meter ese formulario dentro de nuestro listado de notas

```
// estamos en notas.blade.php
```

```
<form action="{{ route('notas.eliminar', $nota) }}" method="POST" class="d-inline">
```

```
    @method('DELETE')
```

```
    @csrf
```

```
    <button class="btn btn-danger btn-sm" type="submit">Eliminar</button>
```

```
</form>
```

Eliminando registros

Ahora que ya tenemos montado el formulario en nuestra plantilla, pasemos a crear la ruta que hemos colocado en el action del formulario para borrar elementos.

```
Route::delete('eliminar/{id}', [ NotasController::class, 'eliminar' ]) ->  
name('notas.eliminar');
```

El último paso que nos queda es modificar el Controller:

```
public function eliminar($id) {  
    $notaEliminar = Nota::findOrFail($id);  
    $notaEliminar -> delete();  
    return back() -> with('mensaje', 'Nota Eliminada');  
}
```

Si todo ha salido bien, habremos creado un sitio en Laravel y Eloquent que es capaz de hacer un CRUD validando campos en formularios e insertando datos reales en una base de datos.

Paginación

Para añadir paginación a nuestros resultados, Eloquent tiene un método que se llama `paginate()` donde le pasamos un número entero como parámetro para indicarle el número de resultados que queremos por página.

```
public function notas() {  
    // $notas = Nota::all();  
    $notas = Nota::paginate(5);  
    return view('notas', compact('notas'));  
}
```

En la vista usamos: `{{ $notas->links() }}`

Ahora veremos ciertos elementos HTML que se han generado en nuestra vista, esto es porque Laravel hace uso de una librería de paginación situada en la carpeta `vendor/laravel/framework/src/illuminate/Pagination`.

Para usar Bootstrap para paginar debemos ir a **App\Providers** y a **ServiceProvider** y añadir en el método **boot()** **Paginator::useBootstrap();** e importar **Paginator**.

Bootstrap + vite



Para trabajar con la parte visual de Laravel podemos interactuar con Bootstrap, pero necesitamos gestionar la dependencia con Vite.

Vite (palabra en francés para "rápido", pronunciado como /vit/, como "veet") es una herramienta de compilación que tiene como objetivo proporcionar una experiencia de desarrollo más rápida y ágil para proyectos web modernos.

`npm i bootstrap --save-dev`

`npm install sass --save-dev`

Abre el archivo vite.config.js que se encuentra en la raíz del proyecto y cambia la referencia resources/css/app.css por resources/css/app.scss

En el archivo resources/css/app.scss importa Bootstrap agregando la siguiente línea de código:

```
@import 'bootstrap/scss/bootstrap';
```

Abre el archivo resources/js/app.js y agrega el siguiente código:

```
import * as bootstrap from 'bootstrap'
```

Para poderlo usar con libertad en nuestras vistas (blade.php)

```
@vite(['resources/js/app.js', 'resources/css/app.scss']) en el <head>
```


Transacciones - Integridad

Para las transacciones de la base de datos, hay tres métodos en la parte de la base de datos para iniciar una transacción manualmente y tener un control total sobre la gestión de las transacciones.

Muchos usuarios encuentran estas opciones más flexibles para definir exactamente cuándo se debe confirmar o revertir una transacción:

- Crea una transacción: Utiliza el comando `DB::beginTransaction();` para iniciar una transacción.
- Deshacer una transacción: Utiliza el comando `DB::rollBack();` si quieres realizar cambios o deshacer acciones.
- Confirmar una transacción: Si todo ha ido según lo previsto, utiliza el comando `DB::commit();`.

Recuerda siempre concluir cada transacción abierta con una acción de consignación o de reversión, especialmente los bucles. De lo contrario, este método manual se desincronizará y tus registros no se actualizarán.

Transacciones - Integridad

Ejemplo:

```
try {  
    DB::beginTransaction();  
    $employee = Employee::create([  
        'name' => $request->employee_name,  
        'email' => $request->employee_email  
    ]);  
    DB::afterCommit(function() use($employee){  
        Mail::to($employee)->send(new EmployeeHired);  
    });  
    $position = Position::create([  
        'name' => $request->position,  
    ]);  
    $position->employees()->save($employee);  
    DB::commit();  
} catch (\Exception $e) {  
    DB::rollBack();  
    return response()->json(['message' => 'Error']);  
}
```

Olga M. Moreno Martín

¿Preguntas?

Olga M. Moreno Martín
Olga M. Moreno Martín

